CI and Pre-Commit Guide

Quality Gates, Code Standards, and Automated CI Pipeline

The Guardian Layer Engineering Team

Version 1.0 | November 2026

1 Purpose

The Guardian Layer codebase enforces automated quality gates at both local developer environments (via git pre-commit hooks) and pull-request integration perimeters (via GitHub Actions workflows). These verification layers ensure zero regressions in formatting, static analysis, type checking, unit/integration testing, and commit conventions before any code merges into the mainline branch.

2 Pre-Commit Hooks

Local commit validation is orchestrated via `.pre-commit-config.yaml`. The hook suite executes automatically during `git commit`:

- **Ruff format:** Enforces PEP 8 code formatting standards across all Python sources.
- **Ruff lint:** Evaluates Python code quality rules, catching dead code, unused imports, and anti-patterns.
- **Tests:** Executes the fast pytest test suite, preventing regression commits.
- **Commitizen:** Validates commit message adherence to Conventional Commits conventions (e.g., `feat:`, `fix:`, `chore:`).
- **Branch check:** Verifies feature and bugfix branch naming standards.

2.1 Installation and Execution

To install hooks locally into the git repository:

```
uv run pre-commit install
uv run pre-commit install --hook-type commit-msg
```

To manually trigger the full pre-commit evaluation against all files:

```
uv run pre-commit run --all-files
```

3 CI Pipeline Architecture

The central continuous integration pipeline is defined in `.github/workflows/ci.yml`. Triggered on every Pull Request targeting the primary branch, the workflow executes the following sequential stages:

1. **Repository Checkout:** Clones repository source with full commit history.
2. **Runtime Provisioning:** Sets up Python 3.12 and resolves frozen lockfile dependencies via `uv sync`.

3. **Linter Validation:** Runs `ruff check` against all codebase modules.
4. **Type Safety Evaluation:** Runs `basedpyright` type analysis, enforcing strict typing contracts.
5. **Test Suite Execution:** Runs all unit and integration tests via `pytest`.
6. **Container Build Check:** Discovers Dockerfiles within repository roots and verifies successful image builds.

Failure policy: If any linter rule, type check, pytest assertion, or container build fails, the CI job fails immediately and blocks branch merging.

4 Docker Compose Environment

The `docker-compose.yml` configuration provides multi-service local infrastructure:

- **Kafka Brokers:** 3-node Apache Kafka cluster using image `apache/kafka:3.8.0` with KRaft consensus.
- **Relational Database:** PostgreSQL 16 container for persistent telemetry state and anomaly verdict tables.
- **Audit Logging:** Elasticsearch 8.15 container storing operational telemetry and forensic logs.

5 Pre-PR Submission Checklist

Before opening a Pull Request, engineers must execute the following local verification sequence:

```
uv run pre-commit run --all-files
git diff --check
git status
```

Ensure all local hooks pass and all changes are cleanly staged before requesting team review.